

งาน: พัฒนาระบบแนะนำเป็นระบบของตัวเอง • รหัส 007

ระบบแนะนำมือถือ

PHONE RECOMMENDER SYSTEM • Neo4j (Graph DB) + Streamlit

ข้อมูลของเราเอง 12 คน $\times$ 23 รุ่น $\times$ 38 ความสนใจ (ไม่ใช่ dataset สำเร็จรูป)

ผลการแนะนำแสดง "ภาพสินค้าจริง" ทุกครั้ง + บอกเหตุผลได้ว่าเพราะใคร/เพราะรุ่นไหน

งานแนวเดียวกับตัวอย่างในวิชา แต่เปลี่ยนโดเมนเป็นมือถือ และใช้สัญญาณ "ชื้อ้อ + ระดับราคา"

12

ผู้ใช้งานระบบ

23

รุ่นมือถือ (11 ยี่ห้อ)

38

ความสนใจ (LIKES)

4

วิธีให้คะแนน + ผสม

สาริตถการใช้งาน: เว็บแอป Streamlit 7 หน้า (Dashboard $\rightarrow$ Admin + หน้ารวมงาน) • ข้อมูลบน Neo4j จริง • โค้ด/สไลด์/โน้ตบุ๊กอยู่บน GitHub

AGENDA

หัวข้อที่จะนำเสนอ

01

โจทย์และสิ่งที่ส่ง

งาน 4 ข้อของอาจารย์ → ส่งครบอะไรบ้าง

02

แนวคิดระบบแนะนำกราฟ

ทำไมข้อมูล “ใครสนใจรุ่นไหน” ต้องใช้กราฟ + การเดิน 3 hop

03

ข้อมูลและสถาปัตยกรรม

ข้อมูลชุดของเรา + Neo4j + Streamlit + GitHub

04

ระบบทำงานจริง

ผลการแนะนำ 5 วิธี + ภาพสินค้าจริง + อธิบายเหตุผลข้อหลังได้

05

สถิติการใช้งาน (Like & Rate)

เพิ่มความสนใจ/ให้ดาว แล้วคำแนะนำเปลี่ยนทันที ไม่ต้องเทรนใหม่

06

การทดสอบและลิงก์งาน

ผลตรวจสอบจริง + Colab + GitHub + หน้า index

งานที่อาจารย์สั่ง 4 ข้อ — ส่งครบทุกข้อ

1

Present ด้วย PowerPoint + สถิติการใช้งานระบบ

สไลด์ 16 หน้า (ไฟล์นี้) + เดโมสไลนแอป Streamlit + ระบบแสดง “ภาพมือถือจริง” ในผลการแนะนำ

2

ข้อมูล PowerPoint ไว้ใน GitHub

ใส่ทั้ง .pptx และ .pdf ไว้ในโฟลเดอร์ slides/ ของ repo (ดูได้โดยไม่ต้องติดตั้งอะไร)

3

เอาการบ้านทุกอันไว้ใน GitHub

ทุกชิ้นงานมี repo ของตัวเอง และมี repo กลาง homework รวมทุกงานไว้ทีเดียว

4

จัดหน้า index ให้ลิงก์กับงานทุกอัน

หน้าเว็บ GitHub Pages ลิงก์ทุก repo พร้อมคำอธิบายและวันที่อัปเดตล่าสุด

ทำไมงานนี้ต้องใช้ฐานข้อมูลกราฟ?

ข้อมูล "ใครสนใจมือถือรุ่นไหน" คือข้อมูลความสัมพันธ์ล้วน ๆ ถ้าเก็บเป็นตารางต้อง JOIN หลายชั้นกว่าจะตอบได้ แต่ในกราฟเขียนครั้งเดียวจบ

คำแนะนำทั้งระบบเขียนด้วย **Cypher** ไม่กี่บรรทัด

```
MATCH (me:User {user: 'สมชาย'})-[:LIKES]->
      (shared:Phone)<-[:LIKES]-(other:User)
MATCH (other)-[:LIKES]->(rec:Phone)
WHERE NOT (me)-[:LIKES]->(rec)
RETURN rec.model, count(DISTINCT other) AS votes
ORDER BY votes DESC
```

ขั้นตอนการเดิน 3 hop

1. ดูรุ่นที่ผู้ใช้สนใจ (4 รุ่นของ สมชาย)
2. หาคำคนอื่นที่สนใจรุ่นเดียวกัน = รสนิยมใกล้เคียงกัน
3. เก็บรุ่นที่คนกลุ่มนั้นสนใจ แต่ผู้ใช้ยังไม่สนใจ
4. ให้คะแนน → เรียงอันดับ → แสดงภาพสินค้า

ทำไมไม่ใช้ตาราง (SQL) เลย ๆ?

- กราฟเดินความสัมพันธ์ได้ลึกไม่จำกัดชั้น โดยไม่ต้อง JOIN เพิ่มทุกชั้น
- ถามแบบ "เพื่อนของเพื่อนสนใจอะไร" ได้ทันที (เส้นทาง 3 hop)
- นับซ้ำให้ง่ายด้วย count(DISTINCT ...) — 1 คน = 1 เสียง
- เพิ่มผู้ใช้/รุ่นใหม่แล้วแนะนำได้ทันที ไม่ต้องเทรนโมเดลใหม่
- อธิบายย้อนหลังได้ว่าคำแนะนำมาจากใครและรุ่นไหน

ข้อแลกเปลี่ยน: ถ้าข้อมูลความสนใจมีหลักล้านเส้น การนับด้วย Cypher ทุกครั้งจะช้ากว่า การ precompute — งานนี้ข้อมูลเล็ก (38 เส้น) จึงตอบได้ทันที

เก็บข้อมูลเองทั้งหมด — ไม่ใช่ dataset สำเร็จรูป

12

ผู้ใช้ (User)

23

รุ่นมือถือ (Phone)

38

ความสนใจ (LIKES)

11

ยี่ห้อ (Brand)

3 ระดับราคา: เรือธง • ระดับกลาง • ระดับเริ่มต้น

ยี่ห้อ: ASUS, Apple, Google, Honor, OPPO, OnePlus, Samsung, Sony, Xiaomi, realme, vivo

คะแนนดาว (RATED {stars}) 38 เส้น • ดาวเฉลี่ยต่อรุ่นใช้เป็นสัญญาณที่ 5

12 ผู้ใช้ แบ่งตามกลุ่มรสนิยม

สมชาย	สาย Android เรือธง
นรินทร์	สาย Android เดิม ๆ (stock)
มะลิ	สาย Apple
ปวีณา	สาย Apple + Samsung
อนุชา	สาย Samsung
พลอย	สายถ่ายรูป (กล้องดี)
กัญญา	สายคุ้มค่า (งบจำกัด)
เฟิร์น	สายดีไซน์ + กล้อง
ชัย	สายเกมมิ่ง
ธนกร	สายสเปกแรง
เบม	สายแบดอิค งบประหยัด
มินต์	สายดีไซน์ไม่เหมือนใคร

ทำไมออกแบบข้อมูลแบบนี้

- แต่ละคนสนใจ 2-4 รุ่น → มีทั้งคนที่รสนิยมซ้ำกันและคนที่ต่างกันสุดขั้ว
- มีกลุ่ม "สายยี่ห้อ" (Apple/Samsung) และ "สายงบ" (รุ่นกลาง) เพื่อทดสอบสัญญาณ Brand/Tier
- ทุกรุ่นมีคนสนใจอย่างน้อย 1 คน → ไม่มีโหนดร้างในกราฟ
- ภาพสินค้าดึงจาก Wikimedia Commons แล้วเก็บไว้ในโปรเจกต์ (แสดงได้แม้ไม่มีเน็ต)

ทุกรุ่นในการ์ดมีภาพสินค้าจริง (ตามโจทย์ข้อ 1)



ภาพถูกใช้ 2 ที่

- ในแอป: การ์ดผลการแนะนำ + คลังรุ่นทั้งหมด
- ในโน้ตบุ๊ก: เซลล์แสดงผลพร้อมภาพทุกครั้ง
- ในสไลด์: หน้าสาริตผลการแนะนำ

ที่มา: Wikimedia Commons (สัญญาอนุญาตเสรี) เก็บชื่อไฟล์ต้นทางใน data/phones.json เพื่อให้เครดิตได้ครบ
— ตรวจสอบภาพว่าไม่ผิดรุ่น ไม่ใช่โลโก้ และไม่ใช้ภาพเปล่า

โครงสร้างกราฟและจำนวนข้อมูลจริงใน Neo4j

Schema ที่โหลดเข้า Neo4j จริง

(User {user, age, group})
| [:LIKES] 38 เส้น



(Phone {phone_id, model, brand, tier, image})
| [:BY_BRAND] → (Brand {name}) 11 โหนด
| [:IN_TIER] → (Tier {name}) 3 โหนด

ทำไมแยก Brand/Tier เป็นโหนด?

- Brand/Tier ใช้เป็นสัญญาณที่สอง (content-based) ไม่ต้องพึ่งเพื่อน
- ช่วยแก้ cold start ของผู้ใช้ใหม่
- จำนวน "ข้อดีเหมือนกันรุ่น" ได้ด้วยการเดินกราฟ ไม่ต้องเขียนโค้ดนับเอง

คำถามหลัก 3 hop ที่ระบบใช้จริง (ผลจากข้อมูลชุดนี้)

Hop 1-2-3 → รุ่นที่เพื่อนสนิทมักสนใจ

```

MATCH (me:User {user: 'สมชาย'})-[:LIKES]->(shared)
MATCH (shared)-[:LIKES]-(other:User)
MATCH (other)-[:LIKES]->(rec:Phone)
WHERE NOT (me)-[:LIKES]->(rec)
RETURN rec.model, count(DISTINCT other) AS votes
ORDER BY votes DESC

```

ผลจริงจากกราฟ (เรียงตามจำนวนคน)

realme C35 2 คน: มินต์, เฟิร์น

Samsung Galaxy S23 2 คน: ปวีณา, อนุชา

ASUS ROG Phone 6 2 คน: ชัย, ธนกร

realme 16 Pro 2 คน: ธนกร, เฟิร์น

Samsung Galaxy A55 1 คน: อนุชา

5 วิธี (เลือกเทียบกันได้ในแอป)

1) นับโหวตเพื่อน

คะแนน = จำนวนคนที่สนใจรุ่นนั้น

`count(DISTINCT other) → 1 คน = 1 เสียง`

ข้อดี: เข้าใจง่าย | ข้อเสีย: ทุกคนน้ำหนักเท่ากัน

2) ถ่วงน้ำหนัก (Jaccard)

คะแนน = $\sum \text{Jaccard}(\text{เรา}, \text{คนอื่น})$

$\text{Jaccard} = |\text{สนใจร่วม}| \div |\text{รุ่นของสองคนรวมกัน}|$

คนรสนิยมใกล้เคียงมีเสียงมากที่สุด

3) ตามยี่ห้อ/ระดับราคา

นับรุ่นที่เราเคยสนใจซึ่งใช้ยี่ห้อเดียวกัน

+ อยู่ระดับราคาเดียวกัน

ใช้ได้กับผู้ใช้ใหม่ (cold start)

4) ผสมสองสัญญาณ

$0.6 \times (\text{คะแนนเพื่อน normalize})$

$+ 0.4 \times (\text{คะแนนยี่ห้อ/ราคา normalize})$

แก้ปัญหา “แนะนำของนอกกอง” เช่นงบกลางแต่ได้เรือธง

5) ถ่วงด้วยคะแนนดาว — $\text{คะแนน} = \sum (\text{Jaccard}(\text{เรา}, \text{เพื่อน}) \times \text{ดาวที่เพื่อนให้รุ่นนั้น} \div 5)$ ใช้ความสัมพันธ์ RATED {stars} → คนที่ให้ 5 ดาวมีน้ำหนักมากกว่าคนที่ให้ 2 ดาว (ทุกวิธีถูกตรวจสอบเทียบกันแล้ว — ดูหน้าผลการทดสอบ)

ภาพรวมระบบ 6 ตัวชี้วัด + กราฟ + โปรไฟล์ผู้ใช้

การคัดค้านะนำมีภาพสินค้า + คะแนน + เหตุผล (5 วิธี)

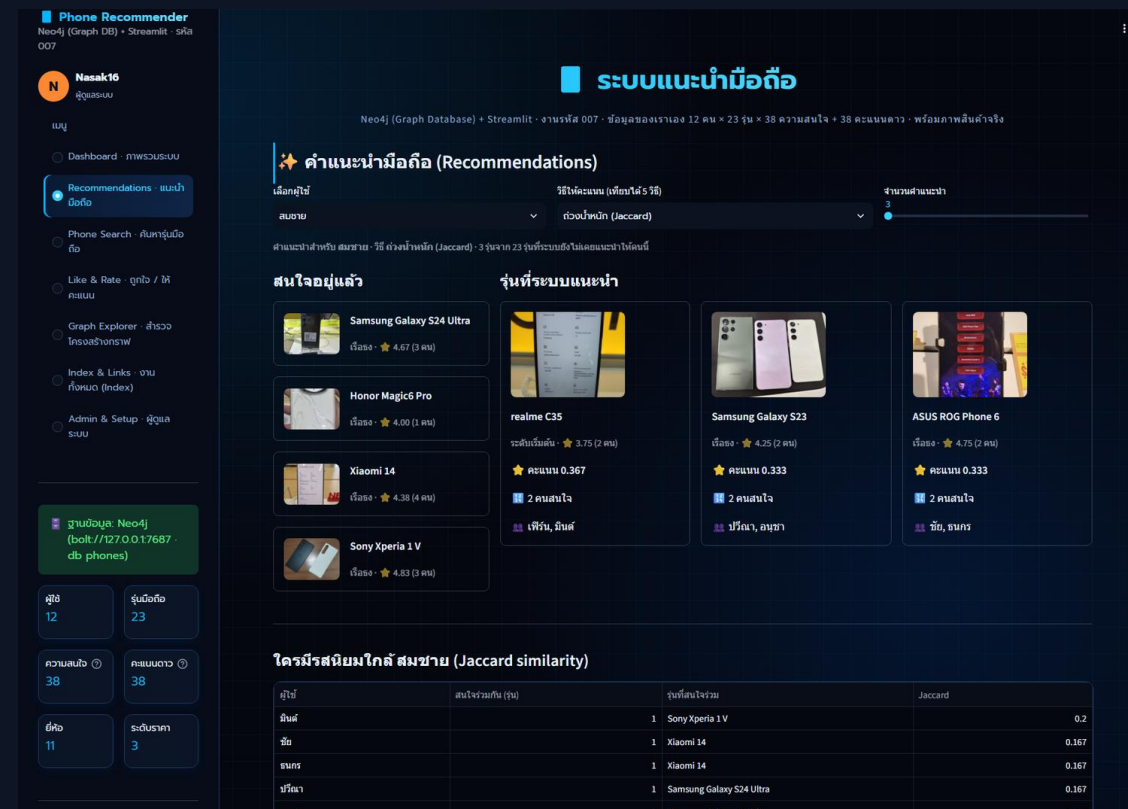
ค้นหา/กรอง 23 วัน พร้อมภาพสินค้าจริง

เพิ่ม/ลบความสนใจ + ให้ดาว แล้วเห็นผลก่อน-หลัง

กราฟความสัมพันธ์ + ตาราง edge + รัน Cypher เอง

schema + วิไลดข้อมูล idempotent + ดาวน์โหลด CSV

หัวข้อรวมงานทกชิ้นบน [GitHub](#) + [ลิงก์หน้า index](#)



ผลจริง: คำแนะนำสำหรับ สมชาย (พร้อมภาพสินค้า)

รุ่นที่ สมชาย สนใจอยู่แล้ว 4 รุ่น



Samsung Galaxy S24 Ultra
เรือธง



Honor Magic6 Pro
เรือธง



Xiaomi 14
เรือธง

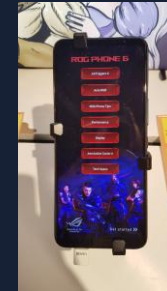


Sony Xperia 1 V
เรือธง



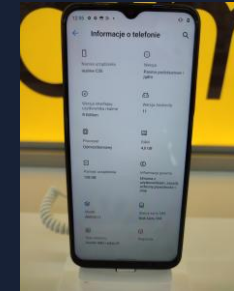
Samsung Galaxy S23
คะแนน 0.944

เพื่อน: ปวีณา, อนุชา
เพื่อนที่รสนิยมใกล้เคียง + ชี้



ASUS ROG Phone 6
คะแนน 0.864

เพื่อน: ชัย, ธนกร
เพื่อนที่รสนิยมใกล้เคียง + ชี้



realme C35
คะแนน 0.6

เพื่อน: มินต์, เฟิร์น
เพื่อนที่รสนิยมใกล้เคียง

ทำไมได้รุ่นนี้ — อธิบายย้อนหลังได้ทุกคำแนะนำ

Samsung Galaxy S23 คะแนน 0.944 | เพื่อน ปวีณา, อนุชา | เพราะสนใจ Galaxy S24 Ultra

ASUS ROG Phone 6 คะแนน 0.864 | เพื่อน ชัย, ธนกร | เพราะสนใจ Xiaomi 14

realme C35 คะแนน 0.6 | เพื่อน มินต์, เฟิร์น | เพราะสนใจ Xperia 1 V

คะแนนผสม = $0.6 \times \text{เพื่อน (Jaccard)} + 0.4 \times \text{ชี้หรือระดับราคา}$ — ตัวเลขจริงจากข้อมูลชุดนี้

ไครรสนิยมใกล้เคียง สมชาย (ตารางจริงจาก Cypher)

Jaccard = $\frac{|\text{รุ่นที่สนใจร่วมกัน}|}{|\text{รุ่นของสองคนรวมกัน}|}$ — ยิ่งใกล้ 1 ยิ่งเหมือนกัน และยิ่งมีน้ำหนักโหวตมาก

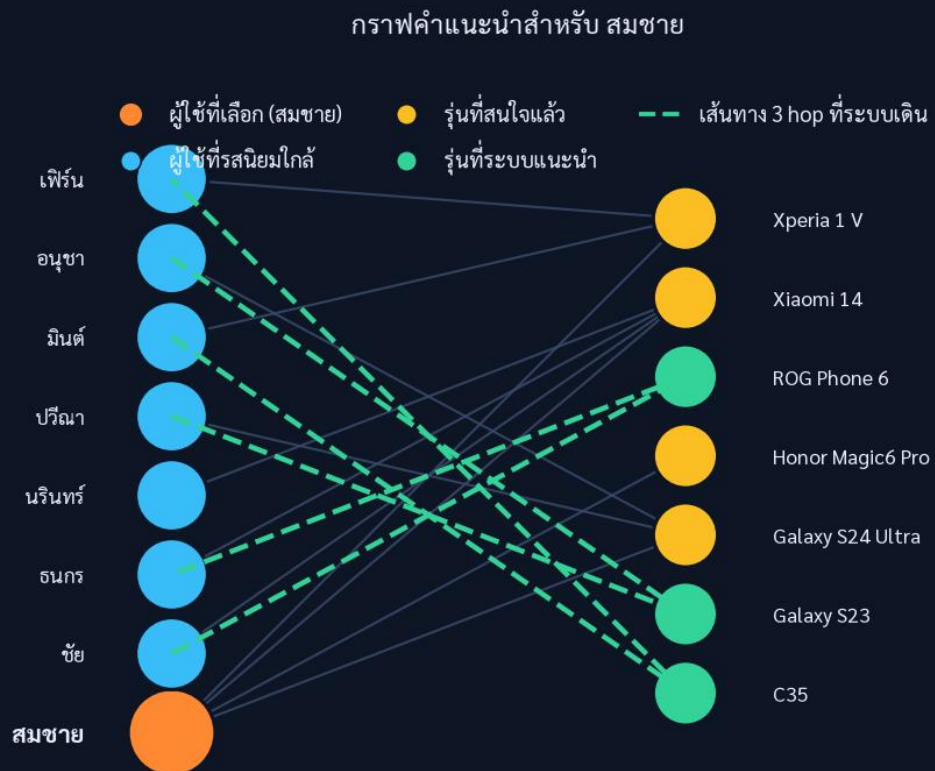
ผู้ใช้ · รุ่นที่สนใจรวม · ค่า Jaccard

มินต์	Sony Xperia 1 V	Jaccard 0.2 (1 รุ่น)
ชัย	Xiaomi 14	Jaccard 0.167 (1 รุ่น)
ธนกร	Xiaomi 14	Jaccard 0.167 (1 รุ่น)
ปวีณา	Samsung Galaxy S24 Ultra	Jaccard 0.167 (1 รุ่น)
อนุชา	Samsung Galaxy S24 Ultra	Jaccard 0.167 (1 รุ่น)

ข้อสังเกต: มี 7 คนที่มีรุ่นสนใจร่วมกับ สมชาย — น้ำหนักโหวตของแต่ละคนไม่เท่ากันตามค่านี้

ทำไมต้อง K จุดนี้: ถ้าใช้แค่ "นับโหวต" คนที่สนใจตรงกันเพียงรุ่นเดียวจะมีเสียงเท่ากับ คนที่รสนิยมเหมือนเราเกือบทั้งหมด — การถ่วงด้วย Jaccard แก้ปัญหานี้

เห็นภาพความสัมพันธ์: กราฟผู้ใช้ ↔ รุ่นมือถือ



อ่านกราฟ

- จุดส้ม = ผู้ใช้ที่เลือก
- จุดฟ้า = คนรสนิยมใกล้เคียง
- จุดเหลือง = รุ่นที่สนใจแล้ว
- จุดเขียว = รุ่นที่ระบบแนะนำ
- เส้นประเขียว = ทางเดิน 3 hop

กราฟสร้างในตัวเอง (ไม่ใช่ภาพนิ่ง) — เปลี่ยนผู้ใช้/วิธีให้คะแนนแล้ววาดใหม่ทันที

Like & Rate: แก่ข้อมูลแล้วคำแนะนำเปลี่ยนทันที

กดเพิ่มความสนใจ 1 รุ่น แล้วดูผลก่อน-หลัง

สิ่งที่แอปทำตอนกดปุ่ม “เพิ่มความสนใจ”

```
// เพิ่มความสนใจใหม่ลงกราฟ
MATCH (u:User {user: 'สมชาย'}), (p:Phone {phone_id: 'P01'})
MERGE (u)-[:LIKES]->(p)

// ให้คะแนนดาวกับรุ่นนั้นด้วย (สัญญาณที่ 5)
MERGE (u)-[:RATED]->(p) SET r.stars = 4.5

// แล้วถามคำแนะนำเดิมซ้ำทันที
// ไม่ต้องเทรนโมเดลใหม่ ไม่ต้องรอ batch
// เพราะคำแนะนำคำนวณสดจากการเดินกราฟ
```

Cold start: ผู้ใช้ใหม่ที่ยังไม่มีเพื่อนรสนิยม

เพิ่มผู้ใช้ใหม่ที่สนใจ Samsung Galaxy S25 (รุ่นที่ยังไม่มีใครสนใจ) → สัญญาณ “เพื่อน” ว้างเปล่า แต่ระบบยังแนะนำได้จากโหนด Brand (Samsung) และ Tier (เรือธง)

ตัวอย่างผลจากสัญญาณซื้อ/ราคา (ผู้ใช้จริง สมชาย)

Samsung Galaxy S23	ซื้อโดยตรง 1 · ราคาตรง 4
OPPO Find X6 Pro	ซื้อโดยตรง 0 · ราคาตรง 4
OPPO Find X7 Ultra	ซื้อโดยตรง 0 · ราคาตรง 4

สะพานเชื่อมกับของเดิม: ระบบนี้ใช้ซ้ำไอเดีย “เพื่อนของเพื่อน” ที่ฝึกในคลาส แต่ย้ายมาคิดบน กราฟทั้งหมด → เขียนสั้นกว่า อ่านง่ายกว่า และเพิ่มสัญญาณซื้อ/ราคาเข้ามาแก้จุดอ่อน เรื่องผู้ใช้ใหม่ได้

ตรวจสอบด้วยข้อมูลจริง ไม่ใช่แค่ “รันผ่าน”

1

ส่ง **backend** ให้ผลตรงกัน

เทียบ Cypher (Neo4j) กับ backend สำรองในหน่วยความจำของผู้ใช้ทั้ง 12 คน × 6 การตรวจ (รวมวิธีถ่วงด้วยคะแนนดาว) — ผลตรงกันครบทุกคน

2

โน้ตบุ๊กรันได้จริงทุกเซลล์

สร้างโน้ตบุ๊กใหม่จากข้อมูลชุดเดียวกัน รันทุกเซลล์บน Neo4j ในเครื่อง มี output จริงฝังในไฟล์ และมี assertion ท้ายไฟล์ยืนยันข้อมูลกลับสู่สภาพเดิม

3

โหมคไม่มี **Neo4j** ต้องไม่พัง

ปิดการเชื่อมต่อฐานข้อมูลแล้วเปิดแอปใหม่ — สลับไปใช้ backend สำรองอัปเดตโน้ต และภาพสินค้าแสดงครบ

4

ภาพต้องถูกต้อง

ตรวจด้วยตา 2 รอบ: รอบแรกพบภาพ iPhone 3 รุ่นมิดเกินไป → หาภาพใหม่จาก Commons แล้วตรวจซ้ำ 23/23 ภาพเป็นตัวเครื่องจริง

งานนี้ทำอะไรได้ และยังสามารถอะไร

ข้อจำกัดที่ควรพูดตอนนำเสนอ

- ข้อมูลความสนใจมี 38 เส้น — น้อย (sparsity) ถ้าเก็บจากผู้ใช้งานจริงจะแน่นขึ้น
- ยังไม่ใช้สเปกเครื่อง (RAM/กล้อง/แบตเตอรี่) เป็นสัญญาณ จึงตอบ “รุ่นไหนกล้องดีที่สุด” ไม่ได้
- ระดับราคาแบ่งหยาบ 3 ระดับ ถ้าใช้ช่วงงบเป็นตัวเลขาจริงจะแนะนำตรงใจกว่า
- ผู้ใช้/ความสนใจเป็นข้อมูลสมมติจากการจัดชุดเพื่อการเรียนการสอน
- คะแนนไม่ได้บอกความมั่นใจ (confidence) ของคำแนะนำ

ต่อยอดได้

- เก็บข้อมูลจริงด้วยแบบสอบถามในห้องเรียน แล้วโหลดเข้าฐานข้อมูลเดิมได้ทันที
- เพิ่มโหนดสเปก (Camera, Battery, Chipset) แล้วเป็นระบบแนะนำแบบหลายเกณฑ์
- เพิ่มน้ำหนักตามความสำคัญที่ผู้ใช้ให้ (เช่น เน้นกล้องมากกว่าราคา)
- เก็บประวัติการกดดู/เทียบรุ่น แล้วเปลี่ยนจาก “ความสนใจ” เป็นพฤติกรรมจริง
- deploy บน Neo4j Aura + Streamlit Community Cloud ให้เปิดลิงก์เดโมได้จากที่ไหนก็ได้

สังครบ 4 ข้อของโจทย์

1

สไลด์นำเสนอ (ไฟล์นี้)

[slides/นำเสนอระบบแนะนำมือถือ_007.pptx + .pdf](#)

2

โน้ตบุ๊กอธิบายวิธีทำ (รันบน Colab)

https://colab.research.google.com/gist/Nasak16/8667b219bbff8253335ec78f78b5c79e/PhoneRecommender_Neo4j_007.ipynb

3

โค้ดระบบทั้งหมดบน GitHub

<https://github.com/Nasak16/phone-recommender>

4

หน้า index รวมงานทุกชิ้น

<https://nasak16.github.io/homework/>

ระบบนี้เป็นของเราเองทั้งหมด: ออกแบบข้อมูลเอง (12 คน $\times$ 23 รุ่น $\times$ 38 ความสนใจ) ออกแบบโครงสร้างกราฟเอง เขียนสูตรให้คะแนน 5 วิธีเอง และแสดงภาพสินค้าจริงในผลการแนะนำ (แอปมี 6 หน้า)

ขอบคุณครับ — ยินดีสาธิตสดในแอปและตอบคำถาม